

Live Coding: Generics copyNumbers

- Generics: Aufgabe 3

Verwende die Klasse **CollectionUtil**.

Implementiere die Methode **static void copyNumbers**, die alle Elemente aus **source** nach **target** kopiert.

Die Methode erhält zwei Parameter **source** und **target**.

Beide Parameter sind von Typ **Collection** und sollen auf **Number** beschränkt sein.

Number ist eine vorhandene Java-Klasse.

Implementiere die Methode einmal nur mit **Wildcards** und einmal **ohne Wildcards**.

Was ist der Unterschied zwischen beiden Varianten?

Schreibe einen Unit Test, der beide Varianten testet.

Wir wollen eine Methode schreiben, die Zahlen von einer Liste in eine andere kopiert. Klingt einfach, oder?

Wir vergleichen zwei Lösungswege für das Kopieren von Listen. Wir zeigen, warum Wildcards hier flexibler sind als ein fester Typparameter `<T>`.

```

final class CollectionUtil {
    public static void main(String[] args) {
        List<Integer> integers = new ArrayList<>(List.of(1, 2, 3)); // Unsere Quelle (Integer)
        List<Number> numbers = new ArrayList<>(List.of(1.1, 2, 3.3f)); // Unser Ziel (Number)
    }
}

```

Wir wollen Zahlen aus einer Liste in eine andere kopieren. Wir haben eine Quelle mit Integers und ein Ziel für Numbers

Erst Grün:

Warum final?

1. **Kein Sinn für Unterklassen:** Eine Utility-Klasse enthält normalerweise nur static Methoden. Sie ist kein Bauplan für Objekte, sondern eher ein "Werkzeugkasten". Es macht logisch keinen Sinn, von einem Werkzeugkasten zu erben (class MyUtil extends CollectionUtil), da es dort nichts zu überschreiben oder zu erweitern gibt, was Instanz-Verhalten betrifft.
2. **Klare Design-Absicht:** Das Schlüsselwort final signalisiert anderen Entwicklern sofort: *"Hier gibt es nichts zu vererben. Diese Klasse ist so, wie sie ist, vollständig."*
3. **Vermeidung von Instanziierung (Zusatz-Tipp):** Oft geht final bei Utility-Klassen Hand in Hand mit einem **privaten Konstruktor**. Wenn man nur statische Methoden hat, will man nicht, dass jemand new CollectionUtil() aufruft.
 - final verhindert extends.
 - private CollectionUtil() {} verhindert new.

Gelb:

Wir wollen Zahlen aus einer Liste in eine andere kopieren. Wir haben eine Quelle mit Integers und ein Ziel für Numbers.

```

static void copyNumbersWildcards(Collection<? extends Number> source, Collection<? super Number> target) {
    target.addAll(source);
}

public static void main(String[] args) {
    List<Integer> integers = new ArrayList<>(List.of(1, 2, 3)); // Unsere Quelle (Integer)
    List<Number> numbers = new ArrayList<>(List.of(1.1, 2, 3.3f)); // Unser Ziel (Number)

    copyNumbersWildcards(integers, numbers);
    System.out.println(numbers);
}

```

Versuchen wir es zuerst mit Wildcards. Erinnert euch an PECS:

Quelle = Producer (extends), Ziel = Consumer (super)."

Warum static?

Weil CollectionUtil eine reine Hilfsklasse (Utility Class) ist. Wir wollen keine Instanz davon erzeugen müssen (new CollectionUtil()), um die Methode zu nutzen. Es verhält sich wie Math.abs(...) oder Collections.sort(...) - einfach Werkzeuge, die man direkt auf der Klasse aufrufen kann."

Die Methodensignatur:

Collection<? extends Number> source: **Producer (Quelle)**. Das ? extends garantiert, dass alles, was wir aus source herausholen, *mindestens* eine Number ist (oder ein Untertyp wie Integer). Wir wissen also sicher: Es kommen Zahlen raus.

Collection<? super Number> target: **Consumer (Ziel)**. Das ? super garantiert, dass die Liste target dazu in der Lage ist, Number-Objekte aufzunehmen. Es

könnte eine `List<Number>` oder `List<Object>` sein. Beides ist sicher.

`target.addAll(source)`: Hier treffen beide Garantien zusammen. `addAll` nimmt Elemente aus der Quelle (die garantiert `Number` sind) und fügt sie ins Ziel ein (das garantiert `Number` aufnimmt). Die Typ-Sicherheit ist perfekt gewährleistet.

Warum Collection statt List?

In der Aufgabenstellung steht `Collection`. Das ist genereller als `List`. Es erlaubt uns, auch `Sets` oder `Queues` zu kopieren. Je allgemeiner der Typ in der Signatur, desto flexibler die Methode.“

Rufe `copyNumbersWildcards(integers, numbers)`; auf

Warum funktioniert der Code?

Stellt euch folgendes Szenario vor: Angenommen `source` ist eine `ArrayList<Double>` und `target` ist eine `ArrayList<Integer>`.

Sowohl `Double` als auch `Integer` erben von `Number`. Aber ein `Double` (aus der Quelle) passt natürlich NICHT in eine `Integer`-Liste (das Ziel)! Das würde crashen.

Auflösung: "Genau davor schützt uns `Collection<? super Number> target`.

Mit dieser Signatur stellt der Compiler sicher, dass `ArrayList<Double>`, `ArrayList<Integer>`, `ArrayList<Long>` usw. **NICHT** als `target` übergeben werden dürfen.

Denn `Integer` ist kein Super-Typ von `Number`. Es muss stattdessen mindestens eine `ArrayList<Number>` als `target` übergeben werden oder etwas 'darüber', z. B. `ArrayList<Object>`.

In solchen Listen passen problemlos `Double`, `Integer`, `Long` usw. rein."

```

static <T extends Number> void copyNumbersGeneric(Collection<T> source, Collection<T> target) {
    target.addAll(source);
}

public static void main(String[] args) {
    List<Integer> integers = new ArrayList<>(List.of(1, 2, 3)); // Unsere Quelle (Integer)
    List<Number> numbers = new ArrayList<>(List.of(1.1, 2, 3.3f)); // Unser Ziel (Number)

    //copyNumbersWildcards(integers, numbers);
    //System.out.println(numbers);

    copyNumbersGeneric(integers, numbers);
    System.out.println(numbers);
}

```

Was wäre, wenn wir statt Wildcards einen festen Typparameter T nehmen? Das sieht man oft fälschlicherweise als erste Idee.

static: Die Methode gehört zur Klasse CollectionUtil und benötigt keine Instanz.

<T extends Number>:

Das <...> vor dem Rückgabetypp (void) definiert einen generischen Typenparameter für diese Methode.

T: Ein Platzhalter-Name (Type).

extends Number: Eine Einschränkung (Bound). T darf nicht irgendwas sein, sondern muss Number oder eine Unterklasse davon sein (z.B. Integer, Double).

void: Die Methode gibt nichts zurück.

Collection<T> source:

Der erste Parameter ist eine Collection, die Elemente vom Typ T enthält.

Wichtig: Wenn T einmal festgelegt ist (z.B. auf Integer), muss diese Collection genau Integer enthalten.

Collection<T> target:

Der zweite Parameter ist *ebenfalls* eine Collection vom Typ T.

Das ist der Knackpunkt: Da hier dasselbe T steht wie bei source, erzwingen wir, dass **beide Listen exakt denselben Typ** haben müssen. Ein Mischen von Integer-Quelle und Number-Ziel ist damit verboten."

Rufe `copyNumbersGeneric(integers, numbers);` auf.

AHA-Moment: Compiler-Fehler! `integers` ist `List<Integer>`, `numbers` ist `List<Number>`. Das verletzt die Regel T source, T target.

```

public static void main(String[] args) {
    List<Number> integers = new ArrayList<>(List.of(1, 2, 3)); // Unsere Quelle (Integer)
    List<Number> numbers = new ArrayList<>(List.of(1.1, 2, 3.3f)); // Unser Ziel (Number)

    //copyNumbersWildcards(integers, numbers);
    //System.out.println(numbers);

    copyNumbersGeneric(integers, numbers);
    System.out.println(numbers);

}

```

Variante Wildcards (?):

Flexibilität: source und target können **unterschiedliche** Typen haben (z.B. Integer -> Number).

Zugriff: source ist Read-Only (Producer), target ist Write-Only (Consumer).

Bedingung: target muss ein valider Container für Number sein.

Variante Generic Method (T):

Starrheit: Beide Listen müssen **dieselben Typen** enthalten (z.B. Integer -> Integer).

Zugriff: Theoretisch können wir beide Listen lesen und schreiben (da wir den exakten Typ T kennen), aber hier schränkt es uns unnötig ein.

Wie lösen? Gleicher Typ siehe gelb.

Dann Main auskommentieren! Vor den Tests:


```

@Test
void copyNumbersWildcards() {
    // Setup: Quelle (Integer) und Ziel (Number mit vorhandenen Werten)
    List<Integer> integers = new ArrayList<>(List.of(1, 2, 3));
    List<Number> numbers = new ArrayList<>(List.of(1.1, 2, 3.3f));

    // Aufruf der Wildcard-Methode (Der flexible Ansatz)
    CollectionUtil.copyNumbersWildcards(integers, numbers);

    // Assert: Wir erwarten, dass die Integers am Ende angefügt wurden.
    // Reihenfolge: Erst die alten Doubles/Floats, dann die neuen Integers.
    List<Number> expected = List.of(1.1, 2, 3.3f, 1, 2, 3);
    assertEquals(expected, numbers);
}

```

Die Aufgabenstellung verlangt Tests. Das ist auch in der Praxis essenziell. Schreiben wir zwei Tests, um das Verhalten zu beweisen.

```

@Test
void copyNumbersGeneric() {
    // Setup: Hier MÜSSEN wir gleiche Typen verwenden, damit es kompiliert!
    List<Integer> integers1 = new ArrayList<>(List.of(1, 2, 3)); // Quelle
    List<Integer> integers2 = new ArrayList<>(List.of(4, 5, 6)); // Ziel

    // Aufruf der Generic-Methode
    CollectionUtil.copyNumbersGeneric(integers1, integers2);

    // Assert: 4, 5, 6 (alt) + 1, 2, 3 (neu)
    List<Number> expected = List.of(4, 5, 6, 1, 2, 3);
    assertEquals(expected, integers2);
}

```

Im zweiten Test (`copyNumbersGeneric`) sehen wir den Zwang zur Gleichheit: Wir müssen zwei `List<Integer>` nehmen. Wir könnten keine `List<Number>` als Ziel übergeben, selbst wenn wir wollten. Das beweist die Einschränkung dieses Ansatzes.

Der Gewinner: Wildcards (?)

Aufruf: `copyNumbersWildcards(List<Integer>, List<Number>)` -> **Klappt!**

Warum:

source (`Integer`) passt in ? extends `Number`.

target (`Number`) passt in ? super `Number`.

Wir können Typen mischen (`Integer` -> `Number`), solange die Richtung stimmt (von speziell zu allgemein).

Fazit: Perfekt für API-Design, wenn man Flexibilität will.

Der Verlierer (in diesem Fall): Generic Method (τ)

Aufruf: `copyNumbersGeneric(List<Integer>, List<Number>)` -> **Fehler!**

Warum:

Java sucht *ein* τ . Ist $\tau = \text{Integer}$? Dann passt das Ziel (`Number`) nicht. Ist $\tau = \text{Number}$? Dann passt die Quelle (`Integer`) nicht (Invarianz!).

Wann funktioniert es? Nur bei `copyNumbersGeneric(List<Integer>, List<Integer>)`.

Fazit: Zu starr für diesen Anwendungsfall. Nutzt τ nur, wenn ihr *erzwingen* wollt, dass Quelle und Ziel exakt denselben Typ haben.